2025 International Conference on Circuit Power and Computing Technologies (ICCPCT)

# FPGA-Based Wild Animal Detection System Using Verilog: Advancing Surveillance and Mitigating Human-Wildlife Conflicts

K.V.Abhishek
Humanitarian Technology (HuT Labs)
Department of Electronics and communication Engineering
Amrita Vishwa Vidyapeetham,
Amritapuri, India
abhishek.kv2004@gmail.com

Rajesh Kannan Megalingam
Humanitarian Technology (HuT Labs)
Department of Electronics and Communication Engineering
Amrita Vishwa Vidyapeetham,
Amritapuri, India
rajeshkannan@ieee.org

2025 8th International Conference on Circuit, Power & Computing Technologies (ICCPCT) | 979-8-3315-4317-4/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICCPCT65132.2025.11176500

***Abstract*— In the context of wildlife monitoring, the challenge of real-time animal detection in surveillance systems has been addressed using FPGA-based hardware acceleration. To this end, this paper proposes a novel system leveraging a modular architecture to enhance detection efficiency while maintaining low latency and power consumption. The design integrates key modules, including edge detection employing Sobel filtering for object boundary enhancement, motion analysis based on frame differencing to identify significant movements, and object classification using predefined size and motion thresholds. A parallel processing strategy is utilized to optimize computational performance. Simulation and validation conducted in ModelSim demonstrate the system's capability to detect and track animal motion efficiently in various scenarios and dynamic conditions. Metrics such as detection responsiveness and computational efficiency evaluate the system's performance. Comparative experiments underscore the scalability and practicality of the proposed FPGA-based approach in addressing human-wildlife conflicts, particularly in conservation areas and public spaces. The results indicate that the system provides a cost-effective and energy-efficient alternative to traditional CPU and GPU solutions for edge computing applications in real-time wildlife monitoring.**

***Keywords*— *Wildlife Detection, FPGA Hardware Acceleration, Human-Wildlife Conflict Mitigation, Wildlife Monitoring System, Wildlife Detection Simulation, Motion-Based Animal Detection.***

## I. Introduction

Human-wildlife conflicts pose a critical challenge in conservation areas, urban peripheries, and public spaces, threatening both human safety and wildlife preservation. Effective monitoring systems are vital for mitigating these conflicts, yet existing solutions predominantly rely on CPU or GPU-based architectures. While these systems offer high performance, they are often costly, power-intensive, and impractical for deployment in remote areas with constrained resources. This study presents an FPGA-based wild animal detection system that utilizes hardware-level parallelism and energy efficiency to process surveillance video feeds, detect animal movement, and generate real-time alerts, enhancing public safety and supporting wildlife conservation with a scalable, cost-effective solution.

By adopting a simulation-based design approach, the system undergoes comprehensive validation without necessitating immediate hardware deployment. This iterative testing framework facilitates refinement, making the system a robust prototype for real-world applications in wildlife conservation and conflict mitigation. The system architecture incorporates modular components optimized for efficiency, scalability, and ease of development. These include edge detection, motion analysis, object classification, and control logic modules, which sequentially process video frames to achieve real-time detection. The edge detection module employs Sobel filtering to enhance object boundaries by highlighting gradients, improving the visibility of moving entities. Motion analysis is performed using frame differencing and background subtraction techniques to isolate dynamic regions while filtering out static areas. The object classification module applies size, shape, and motion thresholds to distinguish animals from irrelevant objects, minimizing false positives and enhancing reliability. Control logic consolidates outputs from all modules, triggering alerts only when detection criteria are met, and activating alarms or notifications accordingly.

To simulate real-time performance, an OpenCV script preprocess surveillance footage into low-resolution grayscale frames, converting them into hexadecimal memory files for dynamic simulation. The edge detection module identifies object boundaries using pixel gradients, while motion analysis highlights movement through frame differencing. The classification module enhances accuracy by applying thresholds to focus on relevant objects. Implemented entirely in Verilog, the modules operate concurrently, synchronized by a global clock, leveraging FPGA's parallel processing for high-speed, low-power performance. Validated using ModelSim, the system ensures functional accuracy, timing efficiency, and responsiveness. Simulation results demonstrate its effectiveness in detecting animal movement, providing a strong foundation for real-world hardware implementation. This FPGA-based system provides a cost-effective, energy-efficient solution suited for remote or resource-limited areas, addressing human-wildlife conflicts with a scalable approach that balances technology and real-world needs for safety and conservation.

## II. Related Works

Field Programmable Gate Arrays (FPGAs) are widely recognized for their speed, energy efficiency, and flexibility

979-8-3315-4317-4/25/$31.00 ©2025 IEEE



Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY WARANGAL. Downloaded on October 01,2026 at 04:52:39 UTC from IEEE Xplore. Restrictions apply.

in real-time image and video processing tasks. Research in FPGA-based wild animal detection draws on significant works that showcase their potential for such applications. The paper [1], propose a modular FPGA architecture integrating image capture, processing, storage, and real-time display. Their system balances functionality and performance, demonstrating its applicability in biomedical imaging and video surveillance. Experimental results validate its functional requirements and processing efficiency, highlighting FPGAs' versatility in addressing complex image processing challenges. The study [2], explore runtime partial reconfiguration to enable dynamic FPGA resource allocation with minimal interruptions. Their method enhances speed and optimizes resource use, proving beneficial for digital signal processing and high-performance computing. The authors, in [3], present an efficient FPGA framework for object detection, integrating feature extraction and detection algorithms. Their system achieves reduced latency, high precision, and optimized hardware utilization, outperforming traditional methods in real-time surveillance contexts. These studies affirm FPGAs' effectiveness in resource-constrained, high-speed image processing scenarios.

The paper [4] describes a system that uses FPGA technology to improve motion detection in automated surveillance. Low latency and high accuracy are prioritized in its design, which combines elements including motion analysis, object tracking algorithms, and image acquisition. Results from experiments show notable speed gains over traditional approaches, emphasizing adaptability and scalability for dynamic scenarios. The study highlights the importance of FPGAs in real-time, reliable security systems and highlights their relevance in applications like automated monitoring and smart cities. In order to manage livestock paper [5] explores automated identification technologies including barcodes and RFID. His approach emphasizes precision, dependability, and simplicity of use while integrating hardware and software solutions. The technology has been tested in real-life scenarios and improves traceability, health monitoring, and agricultural productivity. The study emphasizes how important technology breakthroughs are for improving agricultural sustainability and simplifying animal management.

The paper [6] introduces a pipeline architecture for edge detection. Operating with a 10-MHz clock and 2-micron CMOS technology, it achieves real-time processing at 200 million additions per second. The architecture balances precision and simplicity, offering high computational efficiency and low latency. Its adaptability to dynamic scenarios highlights its potential in advancing edge detection and real-time image analysis. The paper [7] demonstrates a real-time video processing solution using FPGA technology. Employing techniques like background subtraction and motion detection, the system efficiently identifies and tracks moving objects in video streams. FPGA's parallel processing ensures high-speed, efficient performance, making it ideal for surveillance, security, and military applications. The study shows that FPGA-based systems meet real-time demands and improve detection reliability over software-based methods. The study [8] presents a multimodal background generation approach for moving object detection. By leveraging FPGA's parallelism, the system achieves low latency and high accuracy across diverse conditions, making it ideal for real-time video tasks like perimeter security and traffic monitoring.

For precise animal recognition and classification in real-time video streams, the paper [9] suggests a framework based on similarity measures. To properly handle occlusions and environmental fluctuations, the system combines machine learning classifiers with image processing computation. Findings show notable gains in recall, precision, and processing speed, which makes it useful for conservation and wildlife monitoring. The publication [10] addresses issues like noise sensitivity and edge detail loss by proposing an enhanced Sobel method with an 8-directional template. This VHDL-based FPGA design improves accuracy by reducing false edges and capturing finer details, offering high speed and hardware efficiency for real-time use. Evaluations show lower gate count, faster processing, and reduced power, making it ideal for VLSI applications. The algorithm's advanced edge detection enhances computer vision, robotics, and medical imaging. It combines hardware optimization and algorithmic improvements, providing key insights for improving edge detection in VLSI and image processing. In many of our earlier work we have addressed research issues related to Embedded systems, wireless technologies, Agriculture technologies, Mobility aids [11-16].

## III. Design and Implementation

The model integrates Python and Verilog frameworks. Video inputs are pre-processed in Python using OpenCV to extract frames and generate grayscale and .mem files. These are then simulated in ModelSim using parallel Verilog modules to produce detection outputs.

### A. System Architecture

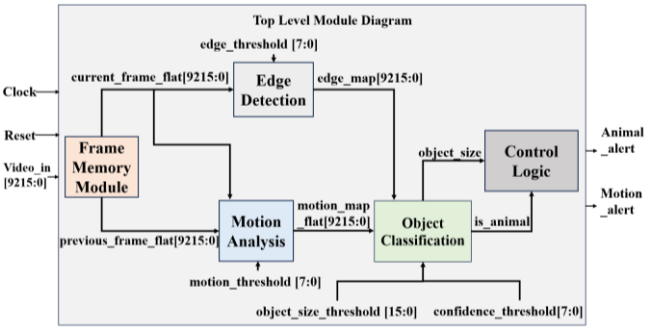



Figure 1. Complete Module Diagram.

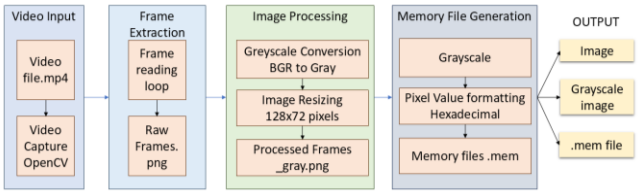



Figure 2. System architecture block representation for video preprocessing using OpenCV and python.

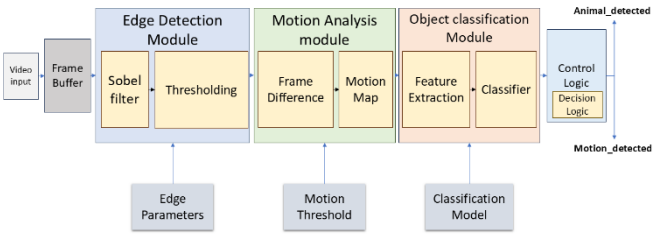



Figure 3. System architecture block representation for the Animal Detection Model.

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY WARANGAL. Downloaded on October 01,2026 at 04:52:39 UTC from IEEE Xplore. Restrictions apply.

There are four essential steps in the system architecture for video preprocessing. The Video Input stage first uses OpenCV's video capture feature to process a .mp4 file. The next step is Frame Extraction, where a loop saves the video's individual frames as raw.png files. These frames are transformed into grayscale and uniformly downsized to 128x72 pixels during the Image Processing step, producing processed.png frames. Lastly, to enable FPGA compatibility, the Memory File Generation stage converts pixel values into hexadecimal and generates .mem files. The outputs guarantee smooth hardware component integration and include the raw image, grayscale image, and matching .mem file.

The system processes video input to detect and classify objects, focusing on animals in surveillance footage. The Frame Buffer stores frames for sequential processing. The Edge Detection Module uses a Sobel filter and thresholding to enhance significant edges, generating pre-processed frames. The Motion Analysis Module compares consecutive frames via frame differencing, creating a motion map to outline changes, with sensitivity controlled by a Motion Threshold. Detected motion triggers the Object Classification Module to extract features and classify objects using a pre-loaded Classification Model. The Control Logic Module evaluates outputs to trigger motion and animal detection alerts, ensuring efficient, real-time performance.

### *B. Algorithm*

The Animal Detection System algorithm processes a sequence of video frames to detect motion and identify animals. It initializes the system by setting the previous frame to a zero matrix. For each video frame, the algorithm calculates the absolute pixel-wise difference between the current and previous frames to generate a binary motion map, where significant motion is indicated by 1 and no motion by 0. The number of motion pixels is then counted, and if this count falls within predefined thresholds, the system identifies the presence of an animal by setting the animal_detected flag to True; otherwise, it remains False. A motion alert is triggered if any motion is detected, while an animal alert is activated if the animal_detected flag is set. After processing a frame, the previous frame is updated to the current frame, and the next frame is loaded. This process repeats for all frames, enabling real-time motion and animal detection for surveillance.

- **Frame Memory Module** : Manages current and previous frames by initializing the previous frame to zero and updating it as new frames are loaded, enabling frame-by-frame motion comparison.
- **Motion Detection Module** : Generates a binary motion map by calculating pixel intensity differences between current and previous frames, marking motion if the difference exceeds a predefined threshold.
- **Object Classification Module** : Analyses the motion map to detect animals by counting motion pixels and comparing their size against thresholds, filtering noise and irrelevant motion.
- **Control Logic Module** : Triggers motion and animal alerts based on outputs from motion detection and object classification, ensuring smooth system communication and output generation.
- **Top Module** : Integrates all submodules, sequentially processes input video frames, and generates final motion and animal detection alerts with relevant metrics.

TABLE I. SIGNAL DESCRIPTION

| Signal | Type | Description |
|---|---|---|
| clock | Input | Clock signal to synchronize all modules. |
| reset | Input | Reset signal to initialize all modules. |
| video_in | Input | Flattened input video frame data. |
| current_frame_flat | Internal | Flattened grayscale pixel data of the current frame. |
| previous_frame_flat | Internal | Flattened grayscale pixel data of the previous frame. Stored for motion comparison. |
| motion_map_flat | Internal | Flattened map of detected motion regions. |
| edge_map_flat | Internal | Flattened image containing edge-detected data. |
| motion_threshold | Input | Defines the minimum pixel intensity change to detect motion. |
| edge_threshold | Input | Defines the minimum contrast level to detect edges. |
| object_size_threshold | Input | Minimum number of connected pixels to classify an object. |
| confidence_threshold | Input | Minimum confidence level required for classification. |
| motion_detected | Internal | High (1) if motion is detected, low (0) otherwise. |
| object_size | Internal | Represents the size of detected objects. |
| is_animal | Internal | High if detected object is classified as an animal. |
| motion_alert | Output | High if motion is detected in the frame. |
| animal_alert | Output | High if an animal is detected in the frame. |

### *C. Circuit/Timing Diagram*

The timing diagrams illustrate the operation of a video processing system that performs motion and animal detection. Running on a 50MHz clock (20ns period), the system processes video frames and generates two key outputs: motion_alert and animal_alert. Each diagram begins with an initialization phase marked by an active-high reset signal. Once reset is deserted, the system begins analysing frames. After reset, the Edge Detection Module processes each frame, applying the Sobel filter and thresholding. When motion is detected by the Motion Analysis Module through frame differencing, the motion_alert signal goes high. If motion exceeds the Motion Threshold, the Object Classification Module activates to analyse the region of interest. If the Classification Model detects an animal, the animal_alert signal is triggered, usually one clock cycle after motion detection due to sequential processing, while the Control Logic Module manages signals based on the analysis.

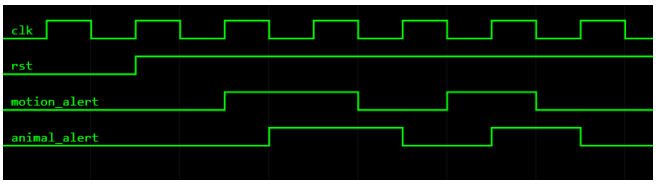



Figure 4. Timing diagram when animal is detected in multiple frames.



Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY WARANGAL. Downloaded on October 01,2026 at 04:52:39 UTC from IEEE Xplore. Restrictions apply.

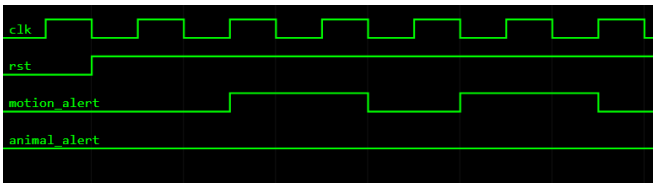


Figure 5. Timing diagram when no animal is detected.

The waveforms show that the system can independently assess scenarios like motion without animals and simultaneous detection, using animal_alert for animal presence and motion_alert for movement detection.

## IV. Experiment and Result

All video files were collected in .mp4 format and pre-processed using Python and OpenCV to extract frames with a resolution of 128x72 (9216 pixels). Eight test cases were created using different CCTV video samples, each representing varying lighting conditions, dynamic environments, noise, and low-quality footage. The pre-processed frames were converted into .mem files for Verilog simulation, with each test case stored in a separate directory named video_x, where x is the test case number.

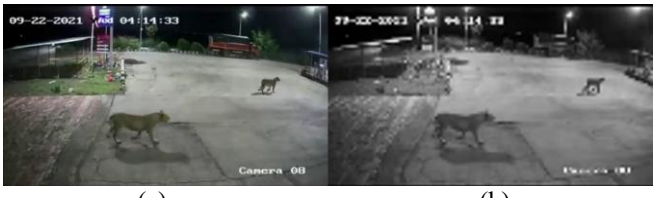
### *A. Testbench*

The testbench is designed to simulate and verify the top module of the video processing system, which performs motion and animal detection on 128×72 resolution frames (9,216 pixels per frame). The system processes 150 frames (adjustable) with 8-bit pixels, running on a 50 MHz clock for efficient ModelSim simulation. A dual-memory testbench stores raw pixel data in video_memory and flattens it in video_in_flat for seamless module integration. Frame loading is managed by the load_frame task, which reads .mem files from designated directories (e.g., C:/intelFPGA/18.1/Project_modules/FPGA/video_x/, where video_x represents each test case. Frame data is processed row by row, converting hex to 8-bit pixels with error handling and debugging by displaying the first 10 pixels per frame. Flattened frame data is then prepared for simulation. The simulation sequence begins with a 50 ns reset period to initialize the system. Flattened frame data is simulated with a 50 ns reset and 1000 ns frame delay, while the testbench logs motion_alert and animal_alert signals to evaluate system performance.

### *B. Simulation Results*

ModelSim simulation of the processed video input generates waveform plots, revealing animal and motion detection patterns.

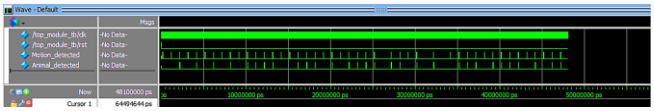
1. Testcase1:



(a) (b)

Figure 6(a). Raw video frame, (b) Corresponding pre-processed grayscale version with two animals in the frame.[17]

Waveform:


Figure 7. Output waveform of testcase1.

The output shows two animals detected in a 3-second video. The motion detection pulse accurately identifies frame-to-frame motion differences, while the animal detection pulse triggers high when the motion map exceeds the threshold, successfully confirming the animals' presence.

2. Testcase 2:


(a) (b)

Figure 8(a). Raw video frame, (b) Corresponding preprocessed grayscale version with one animal in the frame. [18]

Waveform:

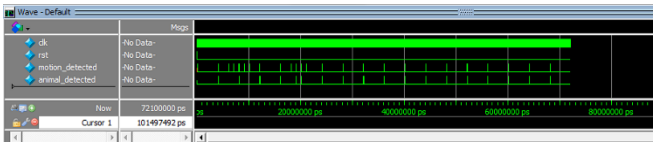


Figure 9. Output waveform of testcase2.

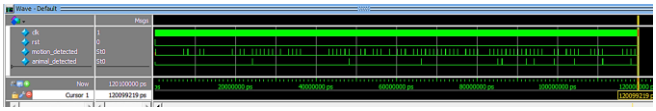
In this 100-frame simulation, the presence of an animal is accurately confirmed when both motion_detected and animal_detected signals are high, indicating activity in a public area. The edge detection effectively distinguishes features even in low-light regions. The detection is highly successful in this test case.

3. Testcase 3:


(a) (b)

Figure 10(a). Raw video frame, (b) Corresponding preprocessed grayscale version with No animal in the frame, but there is presence of shadow near vehicle. [19]

Waveform:


Figure 11. Output waveform of testcase3.

In this test case, although no animal is present, slight movements and shadows near the vehicle and surrounding objects trigger detections in a few frames, highlighting the model's sensitivity to subtle motion cues. Implementing thresholding and specific frequency of detected pulses effectively helps minimize false positives in such scenarios.



Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY WARANGAL. Downloaded on October 01,2026 at 04:52:39 UTC from IEEE Xplore. Restrictions apply.

4. Testcase 4:




(a) (b)

Figure 12(a). Raw video frame, (b) Corresponding preprocessed grayscale version with one animal in the frame, but the lighting is very low creating noise in the greyscale frame. [20]

Waveform:

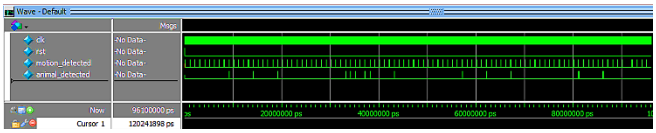


Figure 13. Output waveform of testcase4.

This test case shows significant movement between frames, with motion detected accurately. However, shadows and uneven lighting create noise in motion mapping, especially at lower resolution. Despite these challenges, the model detects animal movement in most frames effectively.

5. Testcase 5:




(a) (b)

Figure 14(a). Raw video frame, (b) Corresponding pre-processed grayscale version with two animals in the frame. [20]

Waveform:

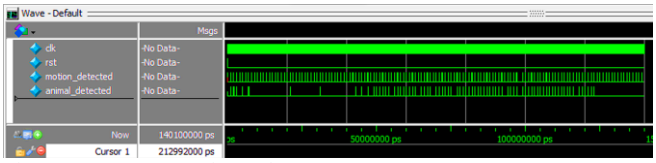


Figure 15. Output waveform of testcase5.

In this test case, it is seen that the animals' i.e. lions' exposure and lighting are better than in test case 4, which leads in more accurate and recurring detection. This shows that the model needs both sufficient lighting and appropriate camera positioning to produce reliable results.

6. Testcase 6:



(a) (b)

Figure 16(a). Raw video frame, (b) Corresponding pre-processed grayscale version with one animal (elephant) in the frame. [21]

Waveform:

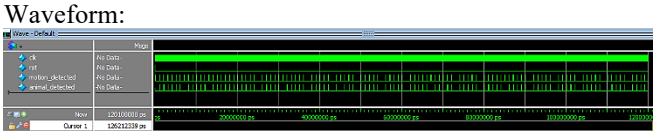


Figure 17. Output waveform of testcase6.

An elephant is visible in the upper corner of the frame in this test case. Despite the challenge of greyscale interpretation, animal_detected signals high with accurate motion detection due to proper motion mapping. This highlights the importance of video quality and lighting for effective detection.

7. Testcase 7:



(a) (b)

Figure 18(a). Raw video frame, (b) Corresponding pre-processed grayscale version with one animal (elephant) in the frame. [22]

Waveform:

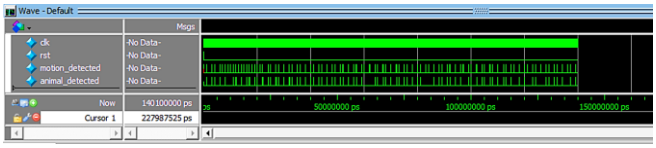


Figure 19. Output waveform of testcase7.

An elephant is visible in the frame in this test case. Even though the situation is extremely difficult the animal_detected is signalling high with a highly accurate output and very accurate motion detection.

8. Testcase 8:




(a) (b)

Figure 20(a). Raw video frame, (b) Corresponding pre-processed grayscale version with humans and an animal in the frame. [23]

Waveform:

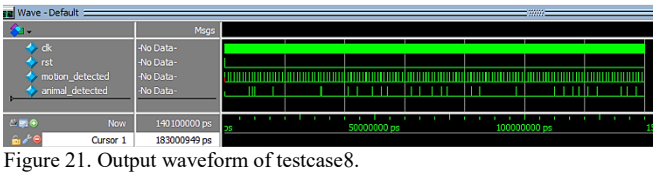


Figure 21. Output waveform of testcase8.

This is a challenging video with a wide range of movements, including those of people, animals, and other things in the environment. Even though it is much harder to tell an animal from a human, the model can still produce a signal with a high degree of accuracy 70% of the time.

## CONCLUSION

This project focuses on simulating an FPGA-based animal identification system using Verilog HDL to detect wild animals in surveillance footage. Leveraging FPGAs' inherent parallelism and high processing speeds makes it well-suited for real-time applications. The simulation employs basic image processing techniques, such as pattern matching to detect unique animal shapes, thresholding for segmentation, and preprocessing to isolate key elements. Emphasis is placed on reducing latency and optimizing computational resources for real-time performance. Simulation findings show precise



Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY WARANGAL. Downloaded on October 01,2026 at 04:52:39 UTC from IEEE Xplore. Restrictions apply.

animal detection against background noise, confirming the efficacy of the selected methods. Since it can watch animal movements and prevent potential hazards, this system has important uses in places where human-wildlife conflicts are common, like wildlife reserves, ranches, and forest perimeters. Its hardware-based design is lightweight and easily incorporated into current security systems. Future improvements, such as more intricate identification algorithms and new animal categories, are made possible by the scalability of the Verilog design. Additionally, this project establishes the foundation for incorporating machine learning models into FPGA-based systems, improving the usefulness and accuracy of detection. This work advances real-time wildlife monitoring technology by bridging the gap between simulation and hardware implementation, providing a scalable and effective solution for conservation efforts and human-animal cohabitation.

## FUTURE WORKS

This study emphasizes simulation-based detection and establishes the foundation for real-time animal detection with FPGA technology. Future developments will use edge detection methods for feature extraction, such as improved Sobel or Canny filters. Rapid identification of wild animals in security footage can be made possible by combining high-resolution cameras with efficient memory modules to enable real-time image processing. This development has the potential to drastically lower the risks and conflicts between humans and animals. In order to improve detection accuracy and adaptability across various species, future work may consider directly implementing machine learning models, such as CNNs, on FPGA. Deploy ability in isolated, remote regions will be ensured by optimizing device design and power usage. Safety in woods and animal areas is increased by the systems' ability to detect detection spots and send SMS alerts to people within a 5-kilometer radius because of the integration of GPS modules. By enabling authorities to track animal movements and take prompt action, this alert system reduces accidents, human-animal confrontations, and gives early warnings—all of which help to conserve wildlife. Future advancements include GSM-based alerts, GPS for location tracking, and improved FPGA design for real-time processing. This promotes ecological conservation and supports human-wildlife coexistence through technological innovation. This device helps with disaster management by identifying environmental changes through animal migrations, reducing conflicts, protecting residential areas and farmlands, and offering a scalable solution for public safety and wildlife monitoring.

## REFERENCES


[1] Zhao Wenge and He Huiming, "FPGA-based video image processing system research," 2010 3rd International Conference on Computer Science and Information Technology, Chengdu, China, 2010, pp. 680-682, doi: 10.1109/ICCSIT.2010.5564541.

[2] S. U. Bhandari, S. Subbaraman, S. Pujari and R. Mahajan, "Real Time Video Processing on FPGA Using on the Fly Partial Reconfiguration," 2009 International Conference on Signal Processing Systems, Singapore, 2009, pp. 244-247, doi: 10.1109/ICSPS.2009.32.

[3] J. Zhao, X. Huang and Y. Massoud, "An efficient real-time FPGA implementation for object detection," 2014 IEEE 12th International New Circuits and Systems Conference (NEWCAS), Trois-Rivieres, QC, Canada, 2014, pp. 313-316, doi: 10.1109/NEWCAS.2014.6934045.

[4] Singh, S.; Shekhar, C.; Vohra, A. FPGA-Based Real-Time Motion Detection for Automated Video Surveillance Systems. *Electronics* **2016**, *5*,10.https://doi.org/10.3390/electronics5010010.

[5] Riordan, Daniel (2012). The design and implementation of animal auto identification systems for application within the agricultural sector. University of Limerick. Thesis. https://hdl.handle.net/10344/7137

[6] N. Kanopoulos, N. Vasanthavada and R. L. Baker, "Design of an image edge detection filter using the Sobel operator," in IEEE Journal of Solid-State Circuits, vol. 23, no. 2, pp. 358-367, April 1988, doi: 10.1109/4.996.

[7] Q. Yi, "FPGA Implementation of Video Capture and Moving Target Detection System," 2018 10th International Conference on Communications, Circuits and Systems (ICCCAS), Chengdu, China, 2018, pp. 373-377, doi: 10.1109/ICCCAS.2018.8769240.

[8] T. Kryjak, M. Komorkiewicz and M. Gorgon, "Real-time moving object detection for video surveillance system in FPGA," Proceedings of the 2011 Conference on Design & Architectures for Signal & Image Processing (DASIP), Tampere, Finland, 2011, pp. 1-8, doi: 10.1109/DASIP.2011.6136881.

[9] S. D. P, T. H P, S. S. N, P. N. Bhat and A. Suresh, "Animal Detection and Classification: a Similarity Measure in Realtime Videos," 2024 2nd International Conference on Networking, Embedded and Wireless Systems (ICNEWS), Bangalore, India, 2024, pp. 1-8, doi: 10.1109/ICNEWS60873.2024.10730999.

[10] G. A. G and R. Ajai A S, "VLSI Implementation of Improved Sobel Edge Detection Algorithm," 2021 International Conference on Communication, Control and Information Sciences (ICCISc), Idukki, India, 2021, pp. 1-6, doi: 10.1109/ICCISc52257.2021.9485022.

[11] R. Kannan Megalingam, S. Kuttankulangara Manoharan, G. Riju and S. Makkal Mohandas, "Netravaad: Interactive Eye Based Communication System for People With Speech Issues," in IEEE Access, vol. 12, pp. 69838-69852, 2024, doi: 10.1109/ACCESS.2024.3402334.

[12] Megalingam, R.K.; Manoharan, S.K.; Mohandas, S.M.; Reddy, C.P.K.; Vijay, E.; Naveen, P.N.V.K.; Chandrika, D. Wearable Hand Orthotic Device for Rehabilitation: Hand Therapy with Multi-Mode Control and Real-Time Feedback. *Appl. Sci.* **2023**, *13*, 3976. https://doi.org/10.3390/app13063976

[13] R. K. Megalingam et al., "Evaluation of Human Intervention-Based Hybrid Approach for Position and Depth Estimation With Error Correction," in IEEE Canadian Journal of Electrical and Computer Engineering, vol. 45, no. 1, pp. 88-98, winter 2022, doi: 10.1109/ICJECE.2021.3134793.

[14] Megalingam, R. K., Vadivel, S. R. R., Rajendraprasad, A., Raj, A., Baskar, S., & Balasubramani Marutha Babu, R. (2022). Development and evaluation of a search-and-rescue robot Paripreksya 2.0 for WRS 2020. *Advanced Robotics*, *36*(21), 1120–1133. https://doi.org/10.1080/01691864.2022.2117575

[15] R. K. Megalingam et al., "Amaran:An Unmanned Robotic Coconut Tree Climber and Harvester," in IEEE/ASME Transactions on Mechatronics, vol. 26, no. 1, pp. 288-299, Feb. 2021, doi: 10.1109/TMECH.2020.3014293.

[16] Megalingam, R.K., Rajendraprasad, A., Raj, A. *et al.* Self-E: a self-driving wheelchair for elders and physically challenged. *Int J Intell Robot Appl* **5**, 477–493 (2021). https://doi.org/10.1007/s41315-021-00209-9

[17] https://youtu.be/K5SxCoTwRaM?si=IXRccLHSyxff4iSb

[18] https://www.youtube.com/watch?v=Wo99ldEMkDs

[19] https://www.youtube.com/watch?v=7TR_CW6wwT8

[20] https://www.youtube.com/watch?v=D4h18ASiT4w

[21] https://youtu.be/eqLK7azDY2o?si=M2WSjlKyUyjrdZ5D

[22] https://youtu.be/MeAVuj_RJ0M?si=KeHWc86OyX58IPd9

[23] https://youtu.be/vWtp7J2jstk?si=3d8O7G-4_uXaCJ-n




Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY WARANGAL. Downloaded on October 01,2026 at 04:52:39 UTC from IEEE Xplore. Restrictions apply.